

UNISENAI JOINVILLE

Curso de Análise e Desenvolvimento de Sistemas (ADS)

Ana Paula dos Reis

**CICLO DE DESENVOLVIMENTO DE SOFTWARE:
as etapas essenciais para a construção de um software de sucesso**

Joinville

2026

Ana Paula dos Reis

**CICLO DE DESENVOLVIMENTO DE SOFTWARE:
as etapas essenciais para a construção de um software de sucesso**

Trabalho acadêmico apresentado ao curso de
Análise e Desenvolvimento de Sistemas da
UniSENAI Joinville, como requisito parcial de
avaliação

Instituição: UniSENAI Joinville

Curso: Análise e Desenvolvimento de Sistemas (ADS)

Aluna: Ana Paula dos Reis

Data: 05/07/2026

Joinville

2026

SUMÁRIO

1 INTRODUÇÃO

2 PROCEDIMENTOS METODOLÓGICOS

3 O CICLO DE VIDA DE DESENVOLVIMENTO DE SOFTWARE (SDLC)

4 MODELOS DE PROCESSO DE DESENVOLVIMENTO DE SOFTWARE

4.1 Modelo em Cascata (Waterfall)

4.2 Modelo em V

4.3 Modelo Espiral

4.4 Modelos Ágeis e o Scrum

5 LEVANTAMENTO DE REQUISITOS

6 ANÁLISE E PLANEJAMENTO

7 DESIGN (DESENHO DA SOLUÇÃO)

8 DESENVOLVIMENTO (CODIFICAÇÃO)

9 TESTES

10 IMPLANTAÇÃO (DEPLOY)

11 MANUTENÇÃO E SUPORTE

12 AVALIAÇÃO E EVOLUÇÃO

13 APLICAÇÃO PRÁTICA: ESTUDO DE CASO ILUSTRATIVO

14 CONSIDERAÇÕES CRÍTICAS SOBRE A ESCOLHA DE MODELOS

15 CONCLUSÃO

REFERÊNCIAS

1 INTRODUÇÃO

Um software raramente fracassa por falta de código. Ele fracassa por falta de processo: requisito mal entendido, prazo estourado, teste que nunca foi feito, versão colocada em produção sem plano de reversão. Por trás de qualquer sistema que funciona bem existe uma sequência de decisões tomadas de forma organizada, e é essa sequência que a engenharia de software convencionou chamar de SDLC — Software Development Life Cycle, ou Ciclo de Vida de Desenvolvimento de Software.

Este trabalho tem como objetivo apresentar as etapas que compõem esse ciclo, discutir os principais modelos de processo que orientam sua execução — cascata, V, espiral e ágil — e demonstrar, por meio de um exemplo prático, como essas etapas se articulam na construção real de um sistema. Para cada etapa do ciclo são descritos o que ocorre, por que ela importa e quais entregas resultam de sua execução, de modo a evidenciar que negligenciar qualquer uma delas compromete o produto final.

A escolha do tema se justifica pela relevância prática da disciplina de Análise e Desenvolvimento de Sistemas: compreender o ciclo de vida do software é pré-requisito para qualquer profissional que pretenda atuar não apenas como codificador, mas como responsável por entregar soluções que atendam de fato às necessidades de quem as utiliza.

2 PROCEDIMENTOS METODOLÓGICOS

O presente trabalho caracteriza-se como uma pesquisa bibliográfica de natureza qualitativa e exploratória. A investigação partiu da leitura de obras de referência sobre engenharia de software e gerenciamento de projetos, como O'Regan (2025) e Reznick e Monson-Haefel (2012), complementadas por artigos técnicos e acadêmicos disponíveis em portais especializados e por publicações institucionais sobre o tema.

A escolha desse tipo de pesquisa se deve à natureza conceitual do assunto: o ciclo de desenvolvimento de software é um objeto de estudo consolidado na literatura, de modo que a contribuição do trabalho está menos em gerar dado inédito e mais em sistematizar, comparar e ilustrar, por meio de um caso hipotético, o funcionamento das etapas e dos modelos de processo mais utilizados pelo mercado.

3 O CICLO DE VIDA DE DESENVOLVIMENTO DE SOFTWARE (SDLC)

O SDLC é um modelo de processo que define, de forma estruturada, as etapas necessárias para desenvolver um software, desde a concepção da ideia inicial até sua implantação e manutenção contínua. Reznick e Monson-Haefel (2012, tradução nossa) resumem que todo programa passa pelas mesmas fases — concepção, levantamento de requisitos, design, codificação, testes, liberação e manutenção —, independentemente do tamanho do projeto ou do número de pessoas envolvidas; o que varia entre os processos é a forma como essas fases se combinam ou se repetem.

O'Regan (2025) reforça essa ideia ao afirmar que a escolha de um ciclo de vida específico depende das necessidades de cada projeto, sendo os modelos cascata, espiral, Processo Unificado Racional (RUP) e Ágil os mais conhecidos na literatura. Segundo o autor, o desenvolvimento de software envolve diversos processos correlatos — definição de requisitos, estimativa, planejamento, projeto, implementação e teste —, e a maturidade desses processos influencia diretamente a qualidade do produto final.

O objetivo do SDLC, portanto, não é impor burocracia, mas garantir que o produto entregue atenda às necessidades de usuários e partes interessadas, respeite prazo e orçamento e apresente qualidade adequada. As seções seguintes detalham primeiro os modelos de processo que organizam essas etapas e, na sequência, cada uma das etapas em si.

4 MODELOS DE PROCESSO DE DESENVOLVIMENTO DE SOFTWARE

Embora as etapas do SDLC sejam relativamente estáveis, a forma como uma equipe as percorre varia conforme o modelo de processo adotado. Um modelo de processo define a ordem, a repetição e o grau de formalidade de cada fase. Os quatro modelos apresentados a seguir —

cascata, V, espiral e ágil — cobrem o espectro que vai do mais rígido e sequencial ao mais iterativo e adaptativo.

4.1 Modelo em Cascata (Waterfall)

O modelo em cascata foi descrito por Winston W. Royce em 1970, em um artigo apresentado na conferência IEEE WESCON, e é considerado o primeiro modelo de ciclo de vida formalmente publicado (ROYCE, 1970). Nele, cada fase — requisitos, projeto, implementação, teste, liberação e manutenção — só é iniciada após a conclusão da anterior, de modo semelhante à queda de água que dá nome ao modelo.

Um detalhe pouco discutido fora do meio acadêmico é que o próprio Royce (1970) já apontava, em seu artigo original, que essa sequência puramente linear era arriscada para sistemas grandes, defendendo a necessidade de retornos entre fases e de um protótipo piloto antes da entrega final. Ainda assim, a versão simplificada do seu diagrama popularizou-se como sinônimo de processo rígido, e é nesse sentido — sequencial, com pouca tolerância a mudanças tardias de requisito — que o termo é empregado até hoje (O'REGAN, 2025).

O modelo funciona bem quando os requisitos são conhecidos e estáveis desde o início, como em projetos de conformidade regulatória ou contratos públicos com escopo fechado. Sua principal fragilidade aparece quando o cliente só percebe um problema de requisito depois que o sistema já está em fase de teste, quando o custo de correção já é alto.

4.2 Modelo em V

O modelo em V é uma variação do cascata que associa explicitamente cada fase de especificação a uma fase de verificação correspondente: o levantamento de requisitos se conecta ao teste de aceitação, a especificação do sistema ao teste de sistema, e assim por diante, formando visualmente a letra V (O'REGAN, 2025). Essa característica o torna especialmente adequado a ambientes de alta criticidade, como sistemas embarcados, financeiros ou de saúde, em que a rastreabilidade entre requisito e teste precisa ficar documentada.

4.3 Modelo Espiral

Proposto por Barry Boehm na década de 1980, o modelo espiral combina elementos do cascata com uma abordagem iterativa orientada a risco. Cada volta da espiral compreende a definição de objetivos, a análise de riscos, o desenvolvimento de um protótipo e a avaliação do cliente antes de planejar a próxima iteração (O'REGAN, 2025). Diferentemente do cascata, o espiral assume que nem todos os requisitos são conhecidos de início e usa o protótipo como ferramenta para reduzir incerteza a cada ciclo.

Esse modelo tende a ser mais adequado a projetos inovadores ou de grande porte, em que o risco técnico é alto e o custo de um erro de arquitetura, descoberto tardiamente, seria proibitivo.

4.4 Modelos Ágeis e o Scrum

Os modelos ágeis surgiram como resposta às limitações do cascata em contextos de requisitos instáveis. Em vez de um único ciclo longo, o trabalho é dividido em iterações curtas — geralmente de uma a quatro semanas —, cada uma produzindo um incremento funcional do software (REZNICK; MONSON-HAEFEL, 2012). O Manifesto Ágil, publicado em 2001, prioriza indivíduos e interações, software funcionando, colaboração com o cliente e resposta a mudanças, sem descartar por completo planejamento e documentação, mas relativizando sua importância diante da entrega contínua de valor.

O Scrum é hoje o framework ágil mais adotado no mercado. Nele, o trabalho é organizado em sprints, com papéis definidos — Product Owner, Scrum Master e equipe de desenvolvimento — e cerimônias como planejamento de sprint, reuniões diárias e retrospectivas. Um comparativo publicado na Revista Ft. observa que, enquanto o modelo cascata privilegia previsibilidade e documentação, os modelos ágeis privilegiam adaptação e proximidade com o cliente, e que a escolha entre eles depende menos de qual é "melhor" e mais do grau de estabilidade dos requisitos do projeto em questão (REVISTA FT., 2022).

Na prática, muitas equipes adotam abordagens híbridas: fases de levantamento de requisitos e arquitetura conduzidas de forma mais tradicional, seguidas de implementação e testes organizados em sprints. As etapas descritas nas seções seguintes existem em qualquer um

desses modelos; o que muda é a ordem, a repetição e o grau de formalidade com que são executadas.

5 LEVANTAMENTO DE REQUISITOS

5.1 Descrição da Etapa

O levantamento de requisitos é a etapa inicial do ciclo de desenvolvimento, na qual a equipe busca compreender as necessidades dos usuários e do negócio que motivam a criação do software. Envolve técnicas como entrevistas, questionários, workshops com partes interessadas e observação de processos já existentes na organização. O'Regan (2025) situa essa etapa como um processo próprio dentro do desenvolvimento, ao lado de estimativa, planejamento, projeto e teste, e destaca que sua qualidade depende diretamente da maturidade dos processos organizacionais empregados.

5.2 Importância

Erros ou omissões no levantamento de requisitos tendem a se propagar por todas as etapas seguintes, tornando-se cada vez mais custosos de corrigir à medida que o projeto avança. Um levantamento malfeito pode resultar na construção da "solução errada, do jeito certo" — um sistema tecnicamente bem construído, mas que não atende à necessidade real do usuário.

5.3 Resultados

O principal produto dessa etapa é um documento de especificação de requisitos (SRS — Software Requirements Specification), contendo requisitos funcionais e não funcionais, que serve de base para as decisões técnicas tomadas nas etapas seguintes.

6 ANÁLISE E PLANEJAMENTO

6.1 Descrição da Etapa

Nesta etapa a equipe define o que será feito com base nos requisitos levantados, estabelecendo escopo, recursos necessários, prazos, orçamento e estratégias a adotar ao longo do

desenvolvimento. É também nesta fase que se realizam as análises de viabilidade técnica, econômica e operacional do projeto.

6.2 Importância

Um planejamento adequado reduz de forma significativa os riscos de atraso, estouro de orçamento e retrabalho. O'Regan (2025) descreve o planejamento como um processo que envolve análise de risco, estimativa de esforço e definição de cronograma, alertando que a estimativa costuma exigir um levantamento razoavelmente detalhado das tarefas antes de ser confiável — o que gera, na prática, uma tensão constante entre a pressão por um cronograma antecipado e a necessidade de detalhar o escopo antes de comprometê-lo. Em contextos ágeis, esse planejamento ocorre de forma distribuída e incremental, com cada sprint contando com seu próprio ciclo de estimativa.

6.3 Resultados

Os principais resultados dessa etapa são um plano de projeto aprovado pelas partes interessadas, contendo cronograma, orçamento estimado, alocação de recursos e identificação preliminar de riscos que possam impactar o desenvolvimento.

7 DESIGN (DESENHO DA SOLUÇÃO)

7.1 Descrição da Etapa

A etapa de design consiste na criação da arquitetura do sistema, incluindo a definição de componentes, interfaces, modelos de dados e a estrutura geral da solução. Envolve também a criação de protótipos e o design de interface com o usuário (UI/UX).

7.2 Importância

Decisões tomadas nessa fase têm impacto direto sobre características de qualidade do software, como manutenibilidade, portabilidade e segurança. É aqui que se definem decisões arquiteturais relevantes, incluindo o tratamento de dados sensíveis desde a concepção — prática conhecida como security by design.

7.3 Resultados

Como resultado, obtêm-se diagramas de arquitetura, protótipos de interface, modelos de dados e demais documentos técnicos (SDD — Software Design Document) que orientam a equipe durante a implementação.

8 DESENVOLVIMENTO (CODIFICAÇÃO)

8.1 Descrição da Etapa

Nesta etapa os desenvolvedores implementam o software conforme planejado nas fases anteriores, escrevendo e compilando o código-fonte da aplicação. É a fase em que o design planejado se transforma em produto funcional.

8.2 Importância

A etapa de desenvolvimento é crítica porque erros de codificação tendem a gerar problemas nas etapas seguintes do ciclo. Reznick e Monson-Haefel (2012) observam que testes isolados costumam identificar cerca de metade dos defeitos de um programa, e que a combinação de testes com revisão de código eleva esse índice para uma faixa entre 93% e 99% — argumento que os autores usam para defender a prática de code review como parte regular do processo de codificação, e não como etapa opcional.

8.3 Resultados

O principal resultado dessa etapa é o código-fonte funcional da aplicação, organizado em repositório de controle de versão, acompanhado de documentação técnica que facilita a compreensão do sistema por outros desenvolvedores.

9 TESTES

9.1 Descrição da Etapa

A etapa de testes verifica se o software funciona corretamente e atende aos requisitos estabelecidos nas etapas iniciais. Envolve diferentes tipos de teste — unitário, de integração, de sistema, de aceitação do usuário, de desempenho e de segurança.

9.2 Importância

Os testes permitem identificar e corrigir falhas antes que o sistema chegue ao usuário final. A identificação precoce de defeitos reduz de forma expressiva o custo de correção, que se torna muito mais elevado quando o problema só é percebido depois da entrega em produção.

9.3 Resultados

Como resultado, obtém-se um relatório de testes (STD — Software Test Documentation), contendo os defeitos identificados e corrigidos, além da validação de que o sistema atende aos critérios de aceitação definidos junto ao cliente.

10 IMPLANTAÇÃO (DEPLOY)

10.1 Descrição da Etapa

A implantação consiste em disponibilizar o software para os usuários finais, envolvendo instalação em ambiente de produção, configuração de servidores, bancos de dados e redes, além da coordenação de recursos necessários para a transição do sistema para uso real.

10.2 Importância

Uma implantação bem planejada reduz indisponibilidades e melhora a experiência do usuário final. É recomendável definir estratégias de reversão (rollback), que permitem retornar o sistema a um estado anterior estável caso surjam problemas durante a implantação.

10.3 Resultados

Os resultados dessa etapa incluem o sistema em pleno funcionamento no ambiente de produção, acompanhado de documentação de suporte e, quando necessário, de treinamento para os usuários do novo sistema.

11 MANUTENÇÃO E SUPORTE

11.1 Descrição da Etapa

A manutenção é a fase que se inicia após a implantação e não possui término definido, persistindo durante todo o período de vida útil do sistema. Envolve correção de defeitos

identificados em produção, implementação de melhorias e adaptação do sistema a novas necessidades tecnológicas, regulatórias ou de negócio.

11.2 Importância

O foco principal dessa etapa é garantir que as necessidades dos usuários continuem sendo atendidas e que o sistema siga funcionando de acordo com as especificações definidas inicialmente. Práticas de monitoramento contínuo permitem identificar gargalos de desempenho e falhas potenciais antes que causem impacto significativo.

11.3 Resultados

Como resultado, tem-se um sistema atualizado e alinhado às necessidades atuais dos usuários, com registro de incidentes resolvidos e de melhorias implementadas ao longo do tempo, além da documentação técnica sempre atualizada.

12 AVALIAÇÃO E EVOLUÇÃO

12.1 Descrição da Etapa

Esta etapa consiste em avaliar os resultados obtidos com o software em produção, coletando métricas de uso, feedback dos usuários e indicadores de desempenho, com o objetivo de planejar evoluções futuras do sistema.

12.2 Importância

A avaliação contínua permite que a organização identifique oportunidades de melhoria e adapte o software às mudanças do mercado e às novas necessidades dos usuários, mantendo sua relevância e competitividade ao longo do tempo.

12.3 Resultados

Os resultados dessa etapa incluem relatórios de desempenho do sistema, indicadores de satisfação dos usuários e um roadmap atualizado com as próximas evoluções planejadas, retroalimentando o ciclo em uma nova iteração.

13 APLICAÇÃO PRÁTICA: ESTUDO DE CASO ILUSTRATIVO

Para tornar concretas as etapas descritas nas seções anteriores, considere o desenvolvimento hipotético de um sistema de agendamento online para uma clínica de pequeno porte, situação comum entre pequenas empresas que buscam digitalizar processos manuais.

No levantamento de requisitos, a equipe entrevista a recepcionista e os profissionais da clínica e identifica que o principal problema não é a falta de um sistema, mas a duplicidade de agendamentos feitos por telefone e por planilha. Esse achado já direciona o escopo: o sistema precisa centralizar os agendamentos, e não apenas digitalizar a planilha existente.

Na análise e planejamento, a equipe opta por um modelo ágil, dado que os requisitos de interface ainda são incertos e a clínica pretende testar o sistema com poucos usuários antes de expandir seu uso. Define-se um cronograma de quatro sprints de duas semanas, priorizando primeiro o cadastro de pacientes e a agenda, depois as notificações automáticas.

No design, a equipe modela o banco de dados relacional para pacientes, profissionais e horários, e produz um protótipo de interface simples, testado informalmente com a recepcionista antes de iniciar a codificação — evitando retrabalho na etapa seguinte.

No desenvolvimento, o time implementa a funcionalidade de agenda na primeira sprint e realiza revisões de código a cada funcionalidade concluída, prática que, segundo Reznick e Monson-Haefel (2012), reduz substancialmente a quantidade de defeitos que chegam à fase de teste.

Nos testes, além dos testes automatizados de unidade, a equipe realiza um teste de aceitação com a própria recepcionista, que identifica um problema real: o sistema não impede o agendamento de dois pacientes no mesmo horário, exatamente o problema original. A correção desse defeito antes da implantação evita que o sistema simplesmente reproduza, em versão digital, a falha que motivou sua criação.

Na implantação, o sistema é colocado em produção primeiro para um único profissional, com um plano de rollback para a planilha caso algo saia errado — o que não chega a ser necessário. Na manutenção, ao longo dos meses seguintes, pequenos ajustes de interface e uma nova funcionalidade de lembrete por e-mail são incorporados a partir do feedback dos usuários, o

que finalmente alimenta a etapa de avaliação, quando a clínica mede a redução de agendamentos duplicados e decide expandir o sistema para os demais profissionais.

O exemplo, ainda que simplificado, evidencia um ponto central deste trabalho: nenhuma etapa funciona isoladamente. O requisito mal levantado no início teria comprometido o teste de aceitação; a ausência de plano de rollback teria tornado a implantação mais arriscada; e a etapa de avaliação só produz informação útil porque houve, desde o início, um objetivo de negócio claro a ser medido.

14 CONSIDERAÇÕES CRÍTICAS SOBRE A ESCOLHA DE MODELOS

Não existe um modelo de processo superior aos demais em termos absolutos. A escolha depende de fatores como estabilidade dos requisitos, criticidade do sistema, maturidade da equipe e restrições contratuais. Um sistema embarcado de uso médico dificilmente se beneficiaria da flexibilidade do Scrum sem os controles de rastreabilidade do modelo em V; por outro lado, um aplicativo de consumo cujos requisitos mudam a cada mês de mercado dificilmente sobreviveria a um ciclo cascata de dois anos.

Vale registrar também uma ressalva sobre a própria origem do modelo cascata: o artigo de Royce (1970) que costuma ser citado como sua fonte já advertia, há mais de cinquenta anos, que um processo puramente sequencial era arriscado para sistemas de grande porte. A rigidez que se associa ao cascata hoje é, em parte, uma simplificação posterior de um argumento que, na origem, já defendia iteração e revisão constante. Esse detalhe é relevante para um curso de Análise e Desenvolvimento de Sistemas porque mostra que a distinção entre "tradicional" e "ágil" é menos absoluta do que a literatura de divulgação costuma sugerir: o problema que a agilidade resolve já havia sido identificado décadas antes do Manifesto Ágil.

15 CONCLUSÃO

A análise realizada ao longo deste trabalho evidencia que o desenvolvimento de um software de sucesso depende da execução cuidadosa de todas as etapas que compõem o seu ciclo de vida, desde o levantamento de requisitos até a avaliação contínua do sistema em produção. Cada etapa desempenha um papel específico e insubstituível, e a negligência em qualquer uma

delas pode comprometer a qualidade, a segurança ou a aderência do software às necessidades reais dos usuários.

A comparação entre os modelos cascata, V, espiral e ágil mostrou ainda que a estrutura das etapas é relativamente constante, mas a forma de percorrê-las varia conforme o grau de incerteza do projeto. O estudo de caso apresentado reforça esse ponto: mesmo em um sistema simples, cada etapa influencia diretamente a qualidade da etapa seguinte, e um erro cometido no início — como um requisito mal compreendido — tende a se manifestar de novo, sob outra forma, se não for corrigido a tempo.

Conclui-se que pensar no ciclo de desenvolvimento de forma integral, e não apenas na etapa de codificação, é o que diferencia um profissional completo de um simples programador de código. Cada etapa faz diferença no sucesso do software, reforçando a importância de uma abordagem estruturada e disciplinada em todos os projetos de desenvolvimento de sistemas.

REFERÊNCIAS

DATAINFO. Ciclo de vida de desenvolvimento de software – o que é? Disponível em: <https://www.datainfo.inf.br/ciclo-de-vida-de-desenvolvimento-de-software-o-que-e/>. Acesso em: 05 jul. 2026.

MICROSOFT. Ciclo de Vida de Desenvolvimento de Software. Power Automate, 2026. Disponível em: <https://www.microsoft.com/pt-pt/power-platform/topics/phases-of-the-software-development-lifecycle>. Acesso em: 05 jul. 2026.

NOVA8 CYBERSECURITY. Entenda o que é SDLC e quais são suas etapas. Disponível em: <https://nova8.com.br/entenda-o-que-e-sdlc-e-quais-sao-suas-etapas/>. Acesso em: 05 jul. 2026.

OBJECTIVE. Modelo Ágil x Modelo Cascata: principais diferenças nas metodologias. Disponível em: <https://www.objective.com.br/insights/modelo-agil-modelo-cascata/>. Acesso em: 05 jul. 2026.

OBJECTIVE. O que é o SDLC e quais suas etapas? Disponível em: <https://www.objective.com.br/insights/sdlc/>. Acesso em: 05 jul. 2026.

O'REGAN, Gerard. Guide to Software Project Management. 2. ed. Cham: Springer Nature Switzerland, 2025. (Undergraduate Topics in Computer Science).

REVISTA FT. Análise comparativa da utilização de metodologia cascata x metodologias ágeis no desenvolvimento de projetos de software. ISSN 1678-0817, 2022. Disponível em:

<https://revistaft.com.br/analise-comparativa-da-utilizacao-de-metodologia-cascata-x-metodologias-ageis-no-desenvolvimento-de-projetos-de-software/>. Acesso em: 05 jul. 2026.

REZNICK, Alan; MONSON-HAEFEL, Richard. Software Development and Professional Practice. New York: Apress, 2012.

ROYCE, Winston W. Managing the Development of Large Software Systems. Proceedings of IEEE WESCON, v. 26, p. 328-388, 1970.

TICOOP BRASIL. Ciclo de Vida do Desenvolvimento de Software (SDLC): etapas, modelos e boas práticas para entregar sistemas com qualidade. Disponível em:

<https://ticoopbrasil.coop.br/ciclo-de-vida-do-desenvolvimento-de-software-sdlc/>. Acesso em: 05 jul. 2026.

WIKIPÉDIA. Ciclo de vida de desenvolvimento de sistemas. Disponível em:

https://pt.wikipedia.org/wiki/Ciclo_de_vida_de_desenvolvimento_de_sistemas. Acesso em: 05 jul. 2026.